

WEEK 4 – DATABASE MANAGEMENT SYSTEM REPORT

SQL TABLES, CONSTRAINTS, RELATIONSHIPS AND REPORTS

In this week's work, I worked with the existing database in Oracle SQL*Plus. I checked the tables, added the required Order_Details records, and used SQL queries to get customer and order information.

1. DATABASE SCHEMA CREATION

I first checked the tables already available in the database and looked at their columns. The main tables used here are Customer, Orders and Order_Details. The Order_Details table contains the details of each item in an order and is connected to the Orders table using Order_ID.

```
CREATE TABLE Order_Details (  
    Order_Detail_ID NUMBER PRIMARY KEY,  
    Order_ID NUMBER,  
    Product_ID NUMBER,  
    Quantity NUMBER,  
    Price NUMBER(10,2)  
);
```

```
65 rows selected.  
QL> SELECT table_name  
2 FROM user_tables  
3 WHERE table_name LIKE '%PRODUCT%';  
  
TABLE_NAME  
-----  
QLPLUS_PRODUCT_PROFILE  
  
QL> SELECT table_name  
2 FROM user_tables  
3 WHERE table_name LIKE '%PROD%'  
4 OR table_name LIKE '%ITEM%';  
  
TABLE_NAME  
-----  
QLPLUS_PRODUCT_PROFILE  
  
QL> SELECT table_name  
2 FROM user_tables  
3 WHERE table_name = 'ORDERS';  
  
TABLE_NAME  
-----  
ORDERS  
  
QL> DESC Orders;  
Name Null? Type  
-----  
ORDER_ID NOT NULL NUMBER(38)  
CUSTOMER_ID NUMBER(38)  
ORDER_DATE DATE  
TOTAL_AMOUNT NUMBER(10,2)  
ORDER_STATUS VARCHAR2(50)  
DELIVERY_ADDRESS VARCHAR2(200)  
  
QL> SELECT constraint_name, constraint_type  
2 FROM user_constraints  
3 WHERE table_name = 'ORDER_DETAILS';
```

2. TABLES AND COLUMNS

The Customer table contains details such as Customer_ID, Full_Name, Email, Phone_Number, Password, Address, City and Pincode. The Orders table contains Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status and Delivery_Address. The Order_Details table contains Order_Detail_ID, Order_ID, Product_ID, Quantity and Price.

```
SQL> DESC Customer;
```

Name	Null?	Type
CUSTOMER_ID	NOT NULL	NUMBER(38)
FULL_NAME		VARCHAR2(100)
EMAIL		VARCHAR2(100)
PHONE_NUMBER		VARCHAR2(15)
PASSWORD		VARCHAR2(100)
ADDRESS		VARCHAR2(200)
CITY		VARCHAR2(50)
PINCODE		VARCHAR2(10)

```
CONSTRAINT_NAME          C
-----
SYS_C007021              P
FK_ORDER                 R
```

```
SQL> ALTER TABLE Order_Details
2 ADD CONSTRAINT chk_quantity
3 CHECK (Quantity > 0);
```

Table altered.

```
SQL> ALTER TABLE Order_Details
2 ADD CONSTRAINT chk_price
3 CHECK (Price >= 0);
```

Table altered.

```
SQL> INSERT INTO Order_Details
2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
3 VALUES
4 (1, 1, 101, 2, 500);
INSERT INTO Order_Details
```

ERROR at line 1:
ORA-02291: integrity constraint (SYSTEM.FK_ORDER) violated - parent key not found

```
SQL> SELECT Order_ID
2 FROM Orders;
```

```
ORDER_ID
-----
1001
1002
1003
```

```
SQL> INSERT INTO Order_Details
2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
```

3. CONSTRAINTS

I used constraints to make sure the data entered into the tables is valid. Order_Details has a primary key and a foreign key that connects it to Orders. There are also CHECK constraints, so Quantity must be greater than zero and Price cannot be negative. Customer_ID in the Customer table is also set as NOT NULL.

```
CONSTRAINT_NAME          C
-----
YS_C007021                P
FK_ORDER                  R

QL> ALTER TABLE Order_Details
  2 ADD CONSTRAINT chk_quantity
  3 CHECK (Quantity > 0);

table altered.

QL> ALTER TABLE Order_Details
  2 ADD CONSTRAINT chk_price
  3 CHECK (Price >= 0);

table altered.

QL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  3 VALUES
  4 (1, 1, 101, 2, 500);
INSERT INTO Order_Details

ERROR at line 1:
ORA-02291: integrity constraint (SYSTEM.FK_ORDER) violated - parent key not
found

QL> SELECT Order_ID
  2 FROM Orders;

ORDER_ID
-----
1001
1002
1003

QL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
```

4. RELATIONSHIPS

The Customer and Orders tables are connected through Customer_ID. The Orders and Order_Details tables are connected through Order_ID. These connections make it possible to get related customer, order and order-item details from the database.

```

SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  3 VALUES
  4 (1, 1001, 101, 2, 500);

1 row created.

SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  3 VALUES
  4 (2, 1002, 102, 1, 750);

1 row created.

SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  3 VALUES
  4 (3, 1003, 103, 3, 250);

1 row created.

SQL> COMMIT;

Commit complete.

SQL> SELECT * FROM Order_Details;

ORDER_DETAIL_ID  ORDER_ID  PRODUCT_ID  QUANTITY  PRICE
-----
                1         1001         101          2       500
                2         1002         102          1       750
                3         1003         103          3       250

SQL> SELECT Order_ID, Order_Date, Total_Amount, Order_Status
  2 FROM Orders
  3 JOIN Order_Details
  4 ON Orders.Order_ID = Order_Details.Order_ID;
SELECT Order_ID, Order_Date, Total_Amount, Order_Status
      *
ERROR at line 1:

```

5. DATA INSERTION

I inserted three Order_Details records using the existing Order_ID values 1001, 1002 and 1003. After entering the records, I used COMMIT and checked the data to make sure it was saved correctly.

```

INSERT INTO Order_Details VALUES (1, 1001, 101, 2, 500);
INSERT INTO Order_Details VALUES (2, 1002, 102, 1, 750);
INSERT INTO Order_Details VALUES (3, 1003, 103, 3, 250);
COMMIT;

```

```

SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  3 VALUES
  4 (1, 1001, 101, 2, 500);

1 row created.

SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  3 VALUES
  4 (2, 1002, 102, 1, 750);

1 row created.

SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  3 VALUES
  4 (3, 1003, 103, 3, 250);

1 row created.

SQL> COMMIT;

Commit complete.

SQL> SELECT * FROM Order_Details;

ORDER_DETAIL_ID  ORDER_ID  PRODUCT_ID  QUANTITY  PRICE
-----
                1         1001         101         2         500
                2         1002         102         1         750
                3         1003         103         3         250

SQL> SELECT Order_ID, Order_Date, Total_Amount, Order_Status
  2 FROM Orders
  3 JOIN Order_Details
  4 ON Orders.Order_ID = Order_Details.Order_ID;
SELECT Order_ID, Order_Date, Total_Amount, Order_Status
      *
ERROR at line 1:

```

6. REPORT 1 – CUSTOMER ORDER HISTORY

This query joins the Customer and Orders tables. It shows the customer's name along with the order ID, order date, total amount and order status. I used FULL_NAME for the customer name.

```

SELECT Customer.FULL_NAME,
       Orders.Order_ID,
       Orders.Order_Date,
       Orders.Total_Amount,
       Orders.Order_Status
FROM Customer
JOIN Orders
ON Customer.Customer_ID = Orders.Customer_ID;

```

Commit complete.

SQL> SELECT * FROM Order_Details;

ORDER_DETAIL_ID	ORDER_ID	PRODUCT_ID	QUANTITY	PRICE
1	1001	101	2	500
2	1002	102	1	750
3	1003	103	3	250

SQL> SELECT Order_ID, Order_Date, Total_Amount, Order_Status
2 FROM Orders
3 JOIN Order_Details
4 ON Orders.Order_ID = Order_Details.Order_ID;

SELECT Order_ID, Order_Date, Total_Amount, Order_Status
*

ERROR at line 1:

ORA-00918: column ambiguously defined

SQL> SELECT Orders.Order_ID, Orders.Order_Date, Orders.Total_Amount, Orders.Order_Status
2 FROM Orders
3 JOIN Order_Details
4 ON Orders.Order_ID = Order_Details.Order_ID;

ORDER_ID	ORDER_DAT	TOTAL_AMOUNT
1001	20-SEP-26	430

Confirmed

1002	21-SEP-26	150
------	-----------	-----

Delivered

1003	22-SEP-26	250
------	-----------	-----

Pending

SQL> SELECT Product_Name, SUM(|

```

SQL> SELECT Customer.FULL_NAME,
2       Orders.Order_ID,
3       Orders.Order_Date,
4       Orders.Total_Amount,
5       Orders.Order_Status
6 FROM Customer
7 JOIN Orders
8 ON Customer.Customer_ID = Orders.Customer_ID;

```

FULL_NAME

ORDER_ID ORDER_DAT TOTAL_AMOUNT

ORDER_STATUS

Ravi Kumar
1001 20-SEP-26 430
Confirmed

Priya Sharma
1002 21-SEP-26 150
Delivered

FULL_NAME

ORDER_ID ORDER_DAT TOTAL_AMOUNT

ORDER_STATUS

Ravi Kumar
1003 22-SEP-26 250
Pending

7. REPORT 2 – PRODUCT-WISE ORDER REPORT

This query groups the order details according to Product_ID. COUNT(*) shows how many order records are present for each product, and SUM(Quantity) shows the total quantity ordered.

```

SELECT Product_ID,
       COUNT(*) AS Number_Of_Times_Ordered,
       SUM(Quantity) AS Total_Quantity_Sold
FROM Order_Details
GROUP BY Product_ID;

```

```

connected.
SQL> SELECT Customer.Customer_Name,
2      Orders.Order_ID,
3      Orders.Order_Date,
4      Orders.Total_Amount,
5      Orders.Order_Status
6 FROM Customer
7 JOIN Orders
8 ON Customer.Customer_ID = Orders.Customer_ID;
SELECT Customer.Customer_Name,
      *
ERROR at line 1:
ORA-00904: "CUSTOMER"."CUSTOMER_NAME": invalid identifier

SQL>

```

```

SQL> SELECT Product_ID,
2      COUNT(*) AS Number_Of_Times_Ordered,
3      SUM(Quantity) AS Total_Quantity_Sold
4 FROM Order_Details
5 GROUP BY Product_ID;

```

PRODUCT_ID	NUMBER_OF_TIMES_ORDERED	TOTAL_QUANTITY_SOLD
102	1	1
101	1	2
103	1	3

```

SQL> SELECT Customer.FULL_NAME,
2      SUM(Orders.Total_Amount) AS Total_Purchase_Amount
3 FROM Customer
4 JOIN Orders
5 ON Customer.Customer_ID = Orders.Customer_ID
6 GROUP BY Customer.FULL_NAME;

```

FULL_NAME	TOTAL_PURCHASE_AMOUNT
Ravi Kumar	680
Priya Sharma	150

```

SQL> |

```

8. REPORT 3 – CUSTOMER SALES ANALYSIS

This query is used to find the total amount spent by each customer. The Customer and Orders tables are joined, and GROUP BY is used to combine the orders belonging to the same customer.

```

SELECT Customer.FULL_NAME,
       SUM(Orders.Total_Amount) AS Total_Purchase_Amount
FROM Customer
JOIN Orders
ON Customer.Customer_ID = Orders.Customer_ID
GROUP BY Customer.FULL_NAME;

```

```

SQL> SELECT Customer.FULL_NAME,
2      Orders.Order_ID,
3      Orders.Order_Date,
4      Orders.Total_Amount,
5      Orders.Order_Status
6 FROM Customer
7 JOIN Orders
8 ON Customer.Customer_ID = Orders.Customer_ID;

```

FULL_NAME

ORDER_ID ORDER_DAT TOTAL_AMOUNT

ORDER_STATUS

```

-----
Ravi Kumar
  1001 20-SEP-26      430
Confirmed

```

```

Priya Sharma
  1002 21-SEP-26      150
Delivered

```

FULL_NAME

ORDER_ID ORDER_DAT TOTAL_AMOUNT

```

SQL> SELECT Product_ID,
2         COUNT(*) AS Number_Of_Times_Ordered,
3         SUM(Quantity) AS Total_Quantity_Sold
4 FROM Order_Details
5 GROUP BY Product_ID;

```

PRODUCT_ID	NUMBER_OF_TIMES_ORDERED	TOTAL_QUANTITY_SOLD
102	1	1
101	1	2
103	1	3

```

SQL> SELECT Customer.FULL_NAME,
2         SUM(Orders.Total_Amount) AS Total_Purchase_Amount
3 FROM Customer
4 JOIN Orders
5 ON Customer.Customer_ID = Orders.Customer_ID
6 GROUP BY Customer.FULL_NAME;

```

FULL_NAME	TOTAL_PURCHASE_AMOUNT
Ravi Kumar	680
Priya Sharma	150

```

SQL> |

```

9. CONCLUSION

I completed the Week 4 SQL work in Oracle SQL*Plus. I created and checked the required table structure, used constraints and relationships, inserted the data, and prepared three reports using JOIN, GROUP BY, COUNT and SUM.